

Rapport SAÉ S2.01 : Base de la programmation objet **- Find My Word**

- **Étudiants** : Abd-Al-Ghani AKLI et Fadel THIAM
- **Établissement** : IUT de Villetaneuse, Université Sorbonne Paris Nord
- **Formation** : BUT Informatique 1ère année

1. Lancement de l'Application (Manuel d'utilisation)

Conformément au cahier des charges, l'application a été conçue pour être exécutée directement depuis un terminal, sans nécessiter d'environnement de développement intégré (IDE).

Instructions :

1. Décompressez les fichiers zip (fichiers_sources_java_S2.01 et wordset_bundle_etudiant) puis placez les fichiers java, le dossier data et le dossier words extrait du fichier jar dans un même répertoire.
2. Ouvrez un terminal et placez-vous dans le répertoire contenant les fichiers sources .java, le dossier data contenant le fichier words.json et le dossier words contenant WordSet et JsonWordSet.
3. Compilez l'ensemble des fichiers avec la commande :
`javac *.java`
4. Lancez l'application via la classe principale :
`java TestGame`

2. Bilan du Projet

2.1. Ce qui a été fait et ce qui fonctionne

- **Logique métier centrale** : L'application respecte les règles strictes de devinette de mots de 5 lettres sans lettres répétées.
- **Validation des saisies** : Le programme vérifie la longueur de la saisie, l'absence de caractères spéciaux (via expression régulière) et l'unicité des lettres. Les entrées invalides ne consomment pas d'essais.
- **Analyse et retours (OK / PRESENT / ABSENT)** : Le cœur algorithmique compare parfaitement la tentative avec le mot secret et renvoie les bons statuts pour l'affichage console.
- **Gestion du score** : Le score dégressif est calculé dynamiquement (de 6 points au premier essai à 1 point au sixième essai) selon la formule $7 - \text{attempts.size}()$.
- **Architecture modulaire** : Le code est strictement divisé en responsabilités uniques (Game, Word, Player, Repositories).

2.2. Ce qui n'a pas été fait ou partiellement fonctionnel

- **Fonctionnalités optionnelles** : La gestion du temps chronométré pour un tour de jeu n'a pas été implémentée afin de privilégier la stabilité du noyau central.
- **Player** : Nous n'avons finalement pas utilisé player pour pouvoir identifier chaque joueur.
- **Historique persistant** : Bien que la grille de la partie en cours s'affiche correctement, l'historique complet entre différentes sessions n'est pas sauvegardé dans un fichier externe.

3. Organisation du Travail en Binôme

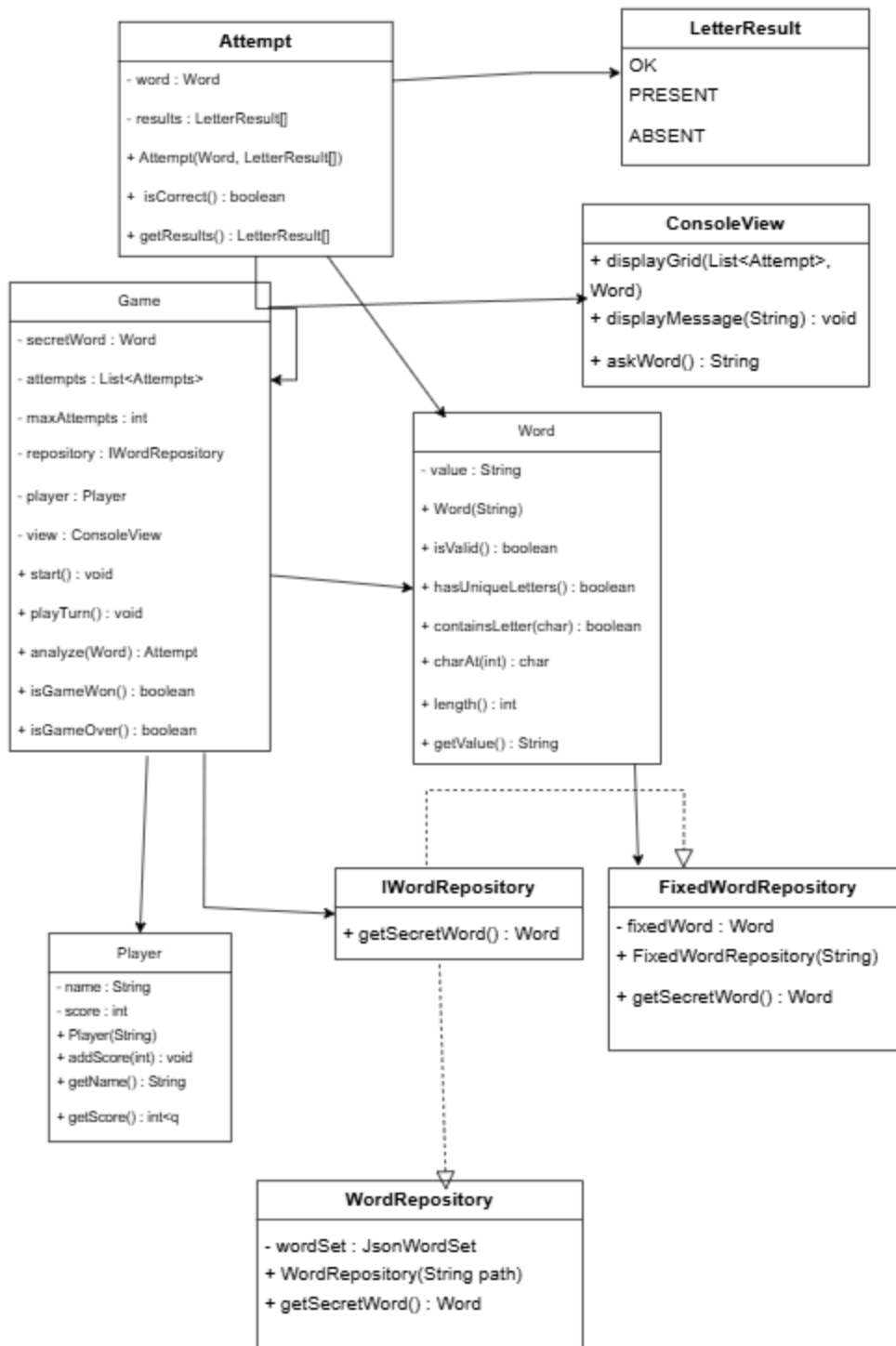
Afin de répondre aux exigences de planification et de travail en équipe, nous avons divisé les responsabilités tout en maintenant une communication constante :

- **Fadel Thiam** : Prise en charge de la modélisation initiale, conception du diagramme de classes UML et définition des interfaces, développement des algorithmes d'analyse (classes Word, Console View, Attempt, Game, IWordRepository, LetterResult).
- **Abd-Al-Ghani Akli** : Implémentation logicielle en Java, développement des algorithmes d'analyse (classe Player, FixedWordRepository, Word Repository).

Cette répartition nous a permis d'avancer en parallèle : la modélisation a guidé le développement, et les contraintes techniques rencontrées lors du codage ont permis d'affiner le diagramme UML de manière itérative.

4. Modélisation UML

Le diagramme ci-dessous présente la modélisation finale qui a servi de plan strict pour l'implémentation Java.



5. Architecture et Justification des Choix Techniques

L'implémentation Java respecte les principes de la programmation orientée objet demandés dans la SAÉ (encapsulation, polymorphisme, séparation des responsabilités).

5.1. Classe Word : Encapsulation des données

Une décision majeure a été de ne pas manipuler des String primitifs, mais de créer une classe métier Word. Cela centralise la logique de traitement :

- **Validation Regex** : La méthode `isValid()` utilise l'expression régulière `[a-zA-Z]+` pour s'assurer que les caractères spéciaux sont rejetés, conformément aux cas de test de l'annexe B.
- **HashSet pour l'unicité** : Pour vérifier qu'un mot ne contient pas de lettres doublons, nous utilisons un `HashSet<Character>`. L'insertion dans un Set se fait en $O(1)$, ce qui rend la validation instantanée et performante.
L'utilisation de cette dernière a nécessité de faire des recherches sur le hachage.

5.2. Polymorphisme avec IWordRepository

L'interface `IWordRepository` permet de découpler le moteur de jeu de la source de données. La classe `Game` ne sait pas si les mots viennent du fichier `words.json` ou d'une liste statique. Cela nous a permis de créer un `FixedWordRepository` pour tester la mécanique (cas de victoire/défaite) sans dépendre de l'implémentation JSON fournie (`JsonWordSet`).

5.3. Le moteur Game et les énumérations

La méthode `analyze()` génère un tableau d'énumérations `LetterResult`. L'utilisation d'énumérations fortement typées empêche les erreurs de code (comparé à l'utilisation de simples entiers comme 0, 1, 2) et facilite grandement la traduction vers l'affichage console dans la View.

6. Difficultés Rencontrées et Solutions Apportées

- **NullPointerException lors des tests** : Au début de l'implémentation, l'instanciation `repository = new FixedWordRepository(null);` provoque un crash lorsque la classe `Word` tentait d'appeler `length()`. **Solution** : Nous avons forcé

l'injection d'une valeur valide lors de l'instanciation et mis en place une meilleure gestion du constructeur de 'Game'.

- **Redondance dans le contrôleur** : La boucle de jeu initiale mélangeait la saisie et la validation. **Solution** : Nous avons encapsulé ces actions dans une méthode `playTurn()` pour alléger la méthode `start()` et respecter le principe DRY (Don't Repeat Yourself).
- **Contrôle des tentatives invalides** : Il a fallu s'assurer que les saisies invalides (trop courtes, caractères spéciaux) ne décrémentent pas le compteur d'essais, comme précisé dans l'Annexe B (Cas 10). La boucle d'acquisition dans la console a été ajustée avec une boucle `while(!word.isValid())` pour bloquer l'utilisateur jusqu'à une saisie conforme.
- **Utilisation du fichier jar** : Nous avons finalement extrait le dossier `wordset` et l'importer dans notre dossier où se trouve notre programme